STUDENT ASSIGNMENT

Deep Learning Foundations | Three Practical Lab Tasks

Purpose: learn which Keras training settings control the optimization rule and the gradient sample size. Use a public dataset and report outcomes you actually compute.

Submission checklist (for each task)

- Executable Python code with per-line explanatory comments.
- A CSV table containing real measured results (not invented metrics).
- Required accuracy comparison graphs with labeled axes.
- A brief discussion explaining what changed, what stayed fixed, and limitations.

Environment

Run in a Google Colab notebook or a Python TensorFlow installation. First-time dataset download requires internet access.

Local setup: pip install tensorflow numpy matplotlib

Lab task 3 | Adaptive optimizers for ANN and MLP

Train a shallow ANN and a deeper MLP using the same public image dataset, swapping only the optimizer constructor while keeping the architecture and fit parameters fixed.

Instructions

1. Dataset MNIST | training 3000 | validation 600 | test 1200.

2. Architecture 1: Flatten → Dense(64) → Dense(10); architecture 2: Flatten → Dense(128) → Dense(64) → Dense(10).

3. Fixed EPOCHS=4 and BATCH_SIZE=64; compare SGD, Adam, RMSprop, Adadelta and Adagrad.

4. Set initial model weights equal across optimizer runs.

For each optimizer, replace only the optimizer constructor and execute the model from fresh initial weights. Use: SGD(0.01), Adam(0.001), RMSprop(0.001), Adadelta(1.0), Adagrad(0.01). These are illustrative trial rates; write the full tf.keras.optimizers.ClassName(learning_rate=...) expression.

Student result table: fill with measured values

Model	Optimizer	Best validation acc.	Test acc.	Seconds
ANN	SGD			
ANN	Adam			
ANN	RMSprop			
ANN	Adadelta			
ANN	Adagrad			
MLP	SGD			
MLP	Adam			
MLP	RMSprop			
MLP	Adadelta			
MLP	Adagrad			

Required discussion (write 150–200 words)

- Which optimizer achieves the highest validation accuracy within each architecture?
- Why is it not justified to claim it always improves results?
- Does using a second hidden layer change the outcome?

Lab task 4 | Adaptive optimizers for CNN

Build one compact convolutional classifier and observe how five Keras optimizers behave on the same images, validation split, initial weights and number of epochs.

Instructions

1. Fashion-MNIST | training 2400 | validation 500 | test 1000.

2. CNN: Conv2D(16) → MaxPool → Conv2D(32) → MaxPool → Flatten → Dense(32) → Dense(10).

3. Fixed EPOCHS=4, BATCH_SIZE=64, identical starting weights and image subsets.

4. Five independent optimizer runs: SGD, Adam, RMSprop, Adadelta and Adagrad.

For each optimizer, replace only the optimizer constructor and execute the model from fresh initial weights. Use: SGD(0.01), Adam(0.001), RMSprop(0.001), Adadelta(1.0), Adagrad(0.01). These are illustrative trial rates; write the full tf.keras.optimizers.ClassName(learning_rate=...) expression.

Student result table: fill with measured values

Optimizer	Best validation acc.	Test acc.	Seconds
SGD			
Adam			
RMSprop			
Adadelta			
Adagrad			

Required discussion (write 150–200 words)

- Which optimizer appears to converge faster on this CNN?
- How do you distinguish accuracy from computation time?
- Why must the input have a grayscale channel?

Lab task 5 | Batch vs stochastic vs mini-batch

Keep the SGD optimizer fixed and change only the fit() batch_size to demonstrate how the same architecture trains with full-batch, single-example, or mini-batch updates.

Instructions

1. Fashion-MNIST | training 600 | validation 200 | test 500.

2. ANN (one hidden layer), MLP (two hidden layers), CNN (two convolution blocks).

3. Keep SGD(learning_rate=0.02), EPOCHS=3 and initial weights fixed within each model.

4. Change BATCH_SIZE only: full batch=600, stochastic=1, mini-batch=32.

Keep optimizer=tf.keras.optimizers.SGD(learning_rate=0.02) unchanged. Change the batch_size argument in model.fit to 600, 1 and 32, respectively. Reset starting weights between trials.

Student result table: fill with measured values

Model	Technique	Batch size	Steps / epoch	Accuracy
ANN	Full batch	600		
ANN	Stochastic	1		
ANN	Mini-batch	32		
MLP	Full batch	600		
MLP	Stochastic	1		
MLP	Mini-batch	32		
CNN	Full batch	600		
CNN	Stochastic	1		
CNN	Mini-batch	32		

Required discussion (write 150–200 words)

- Which technique uses the most updates per epoch, and why?
- How do noisy steps affect observed performance?
- Why are three equal-epoch runs not a matched-update comparison?

General assessment criteria

Criteria	Marks out of 20
Data and model setup	5
Controlled training comparison	5
Plots and measured result table	5
Code commentary and analysis	5

Remember these two one-line controls

Change the update rule: optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

Change how many examples per update: model.fit(..., batch_size=32)

Important: a student should never invent performance metrics. Algorithms may have different outcomes depending on learning rate, seed, data and compute.

Documentation

Keras optimizers reference: https://keras.io/api/optimizers/

TensorFlow Fashion-MNIST tutorial: https://www.tensorflow.org/tutorials/keras/classification

TensorFlow CNN tutorial: https://www.tensorflow.org/tutorials/images/cnn

Google Machine Learning Crash Course: https://developers.google.com/machine-learning/crash-course/linear-regression/hyperparameters